

Hit Rate Is Not Value: A Rigorous Evaluation of Learned and Value-Aware Eviction for Tiered LLM KV-Cache Persistence

Aditya Lawankar

Abstract

Large Language Model serving is bottlenecked by memory capacity for Key-Value (KV) caches. Multi-turn conversations force frequent cache eviction and costly recomputation (the “cold-start” problem). We build a three-tier KV cache persistence system (VRAM $\rightarrow$ NVMe $\rightarrow$ object storage) and evaluate six eviction policies—LRU, a recency heuristic, learned binary classification, static value-density, and a lifetime-normalized space-time density policy—under a statistically rigorous protocol: virtual-clock trace replay, persona-driven synthetic workloads, tier-discounted value accounting, and paired comparisons across 10 seeds with 95% confidence intervals.

Our evaluation yields four findings. (1) *LRU is a far stronger baseline than commonly assumed*: under honest accounting it wins both hit rate and dollar value on every capacity-constrained workload we test, with all paired deltas significant. (2) *Hit rate and value decouple*: on heavy-tailed workloads a simple heuristic retains 97.5% of LRU’s GPU-dollar value while conceding 12 hit-rate points, and policy rankings by hit rate invert under value—our learned classifier beats space-time density by 9 hit-rate points yet delivers \$46/day *less* value. (3) *Static value-density eviction collapses*: maximizing value-per-byte without bounding sojourn time violates Little’s Law dynamics, and our lifetime-normalized space-time objective recovers \$94/day over it (95% CI [77, 112]). (4) A break-even analysis shows tiered persistence is latency-positive from $N^* \approx 12$ tokens for 70B-class GQA models but only beyond $\sim 43k$ tokens for 7B MHA models—the KV-bytes-per-FLOP ratio, not model quality, governs deployability. We validate end-to-end with TinyLlama-1.1B, demonstrating $48\times$ TTFT improvement on CPU with byte-identical greedy decoding from restored caches.

Keywords: Cache Replacement Algorithms, LLM Inference, Tiered Storage, KV Cache, ML Systems

1 Introduction

The cold-start problem in LLM inference occurs when a user resumes a session whose KV cache has been evicted from VRAM. The system must reprocess the entire prompt history through the transformer’s prefill phase, wasting compute and increasing time-to-first-token (TTFT). Existing systems like vLLM [5] rely on reactive LRU eviction, discarding valuable context without regard to recompute cost.

We explore tiered persistence (VRAM $\rightarrow$ NVMe $\rightarrow$ Object Storage) combined with predictive eviction. By proactively migrating idle sessions to cheaper tiers, we hypothesized that a learned eviction policy would improve cache hit rates over LRU. *The data falsified this hypothesis*—and diagnosing why became the contribution: eviction under capacity constraints is a packing problem, a cache is a dynamic system governed by Little’s Law rather than a static knapsack, and hit rate misprices heavy-tailed workloads whose items have quadratically varying recompute costs.

This paper makes the following contributions:

1. A **three-tier KV cache persistence system** with plug-gable eviction backends (LRU, heuristic, learned classification, value-density, space-time density) and a **rigorous evaluation protocol**: virtual-clock replay, persona-driven workloads, tier-discounted value accounting, and paired multi-seed statistics.
2. Evidence that **LRU is a stronger baseline than the learned-caching literature assumes**: it wins both hit rate and dollar value on every constrained workload we test (all paired deltas significant at 95%).
3. The demonstration that **hit rate and cache value decouple** on heavy-tailed workloads—policy rankings invert between the two metrics—so hit rate alone is the wrong target for KV-cache eviction research.
4. The discovery that **static value-density optimization collapses cache cardinality** under Little’s Law dynamics, and an implemented **space-time density policy** (an LHD-style objective adapted to quadratic recompute costs) that significantly recovers value over it.
5. An analytic **break-even model** for when restoration

beats recomputation, showing N^* varies by four orders of magnitude with model architecture.

2 Background

2.1 Self-Attention and the KV Cache

The self-attention mechanism in transformer-based LLMs attends to all previous tokens. The prefill phase grows quadratically $O(N^2)$ with sequence length N , while KV cache memory grows linearly $O(N)$. Inference engines cache key and value tensors in VRAM; however, VRAM is severely constrained, necessitating eviction policies when multiple sessions compete.

2.2 Bélády’s Algorithm

The provably optimal offline eviction policy [2] evicts the item whose next access occurs furthest in the future: $\arg \max_i \text{next_access}(i)$. For uniform-size, uniform-cost items, LRU is an effective online proxy by assuming temporal locality.

2.3 Weighted Caching

When items have varying sizes and costs, the uniform assumption fails. The problem maps to a dynamic Fractional Knapsack. As formalized by Young [11] and made practical by GreedyDual-Size [3], cost-aware eviction must account for value density per byte. LHD [1] further showed that for *dynamic* caches the correct denominator is not size alone but $\text{size} \times \text{expected lifetime}$ —“hit density.” In KV caching, “value” is the quadratic GPU compute saved by avoiding a cold-start prefill, and “weight” is the linear VRAM consumed; our results (§9) empirically rediscover why the static form of this objective fails and the lifetime-normalized form is required.

3 Related Work

LLM serving systems. PagedAttention [5] partitions the KV cache into fixed-size blocks, eliminating fragmentation and enabling memory sharing. vLLM manages VRAM dynamically but relies on reactive LRU eviction. CachedAttention [4] persists KV caches across multi-turn conversation rounds through a hierarchical cache, though its focus is architectural integration rather than learned eviction policies.

LMCache [7] and CacheBlend [10] focus on cross-request prefix sharing—caching system prompts shared across users. Our work targets single-session persistence: saving user-specific conversational state over long time

Table 1: Tier transition pipeline

Transition	Serialization	Compression	Rationale
Hot → Warm	Raw Binary	None	Minimize latency
Warm → Cold	Raw Binary	Zstd (L3)	Optimize capacity
Cold → Warm	Raw Binary	Zstd decomp.	Async prefetch

horizons, where the challenge is predicting which sessions will return.

Mooncake [8] proposes a disaggregated KV cache architecture using a tiered hierarchy across nodes. Our work shares the tiered storage motivation but targets the algorithmic eviction policy layer. Where Mooncake asks “how do we distribute cache across a cluster?”, we ask “which entries should we keep when capacity is constrained?”

Token-level KV compression. H2O [12] and Infini-Gen [6] evict or offload individual *tokens* within a sequence’s KV cache. This is orthogonal to our granularity: we decide which *sessions* to retain across tiers, and token-level compression could be composed with our policies to shrink what we store.

Learned cache eviction. LRB [9] learns to approximate Bélády’s oracle for CDN caching; LHD [1] maximizes hits per unit of space-time occupied. Our work differs in the value model: CDN objects have roughly uniform miss costs, whereas KV cache entries have *quadratically* varying recompute costs, which changes both the correct objective (§7.1) and, as we show, the correct evaluation metric.

4 System Design

Our architecture introduces a TieredCacheManager orchestrating three storage tiers:

1. **Hot Tier (VRAM):** In-memory storage. $\sim 0.1\text{ms}$ latency, 50MB capacity.
2. **Warm Tier (NVMe SSD):** Filesystem-backed with memory-mapped reads for files $> 10\text{MB}$. $\sim 10\text{ms}$ latency, 150MB capacity.
3. **Cold Tier (S3/MinIO):** Object storage with Zstandard compression. $\sim 100\text{ms}$ latency, 300MB capacity.

The system exposes three operations: `save` (serialize and write to hot tier with cascading eviction), `load` (search tiers in order, deserialize on hit), and `demote/promote` (explicit tier migration).

Serialization uses a custom binary format with a 64-byte header (magic bytes, version, layer count, tensor dimensions, dtype) followed by packed FP16 tensor data. This is $3\text{--}5\times$ faster than `safetensors` for hot $\leftrightarrow$ warm transfers.

5 Eviction Policy V1: Binary Classification

Our initial approach formulated eviction as supervised ML: predict $P(\text{resume})$ that a session will be accessed again within 1 hour. We engineered 9 features capturing temporal patterns, access behavior, user history, and session characteristics, and trained Logistic Regression and Gradient Boosted Trees on 7-day persona-driven traces (80/20 split). Test AUC is 0.70 (logistic) and 0.73 (GBT); we use the logistic model in the main study for interpretability—its strongest positive coefficient is the user’s historical return rate (+0.75), confirming the personas carry learnable signal. Candidate sets are scored in a single batched prediction per eviction sweep.

Negative Result: Under enterprise load, Logistic V1 achieves **67.9%** hit rate [66.8, 69.0] versus LRU’s **79.2%** [75.9, 82.4]—an 11-point deficit that also translates to \$230/day less GPU value (paired 95% CI [−323, −138]; Table 2).

6 Failure Analysis

This exposed a fundamental flaw: **eviction under capacity constraints is not a classification problem; it is a Knapsack variant**. LRU accidentally approximated correct behavior by rotating entries uniformly. Our ML models confidently kept large “high probability” sessions that choked capacity, forcing out dozens of smaller entries. The binary criterion $\min P(\text{resume})$ ignores both entry size and variable recompute cost.

7 Eviction Policies V2 & V3: Value Density and Space-Time Density

We reformulated the objective based on weighted caching theory:

$$\text{Value Density} = \frac{P(\text{resume}) \times \text{RecomputeCost}(N)}{\text{SizeBytes}(N)} \quad (1)$$

Because prefill scales $O(N^2)$ but memory scales $O(N)$, longer contexts have exponentially higher value density. We also implemented an **Admission Control Gate**: sessions below a value-density threshold are denied entry to the warm tier.

7.1 Space-Time Density: Why Static Knapsack Fails

Our evaluation revealed that static value-density collapses in practice (§9). A cache is not a static knapsack—it is

a dynamic system governed by Little’s Law. Each entry occupies *space over time*. The correct objective must account for expected sojourn time $\mathbb{E}[\Delta t]$:

$$\text{Space-Time Density} = \frac{P(\text{resume}) \times \text{RecomputeCost}(N)}{\text{Size}(N) \times \mathbb{E}[\Delta t]} \quad (2)$$

Under Little’s Law, $L = \lambda \times W$, where L is steady-state items, λ is arrival rate, and W is average sojourn time. Maximizing Value/Byte without bounding W allows large sessions to accumulate unbounded sojourn time, reducing L (cardinality) and collapsing hit rates.

This objective is structurally an adaptation of LHD’s hit density [1]—expected hits per unit of size $\times$ lifetime—to the KV-cache setting. The adaptation is non-trivial in two respects: the numerator weights each hit by a *quadratic* recompute cost rather than counting hits uniformly, and $P(\text{resume})$ comes from a learned session-level model rather than age-binned statistics.

V3 Implementation. We estimate $\mathbb{E}[\Delta t]$ per session as an exponential moving average ($\alpha=0.3$) of observed inter-access gaps, with a 2-hour prior for sessions with no observed gap. Because an entry idle *longer* than its typical rhythm should expect a longer remaining wait, not a shorter one (the inspection paradox), we score with $\max(\text{EMA gap}, \text{current idle time})$. Candidate sets are re-scored on each eviction sweep.

8 Experimental Setup

We evaluated eviction policies using a synthetic workload simulator with three profiles via Poisson arrivals and Log-normal token distributions:

- **Casual:** 50/hr arrival, $\mu=512$, $\sigma=0.8$, low return probability.
- **Enterprise:** 200/hr, $\mu=2048$, $\sigma=0.5$, high return probability.
- **Power User:** 80/hr, $\mu=8192$, $\sigma=1.2$, medium return probability.

User Personas. Each simulated user carries a persistent persona—a return-propensity multiplier, a verbosity scale, and a 10-hour diurnal activity window—drawn once and reused across all of that user’s sessions. Without personas, per-user features such as historical return rate are i.i.d. noise by construction and no predictor can exceed chance (we measured AUC 0.56 under an i.i.d. pool vs. 0.70 with personas). Conversations grow monotonically: each resume appends new turns to the previously cached context.

Virtual Clock. Traces are replayed on a simulated clock advanced to each event’s timestamp. All components—the cache manager, eviction policies, and feature extraction—read time exclusively from this clock. This is essential for validity: replaying a 6-hour trace at

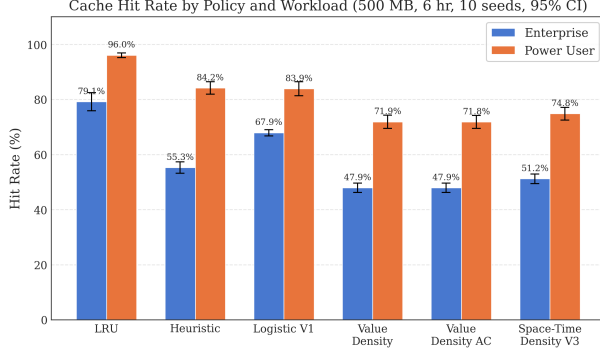


Figure 1: Cache hit rate by policy and workload with 95% CI error bars across 10 seeds. LRU dominates both constrained workloads.

wall-clock speed collapses every temporal feature (session age, idle time, hour of day) to near-zero, silently invalidating any policy that reasons about time while leaving order-based policies like LRU intact.

Model Configuration. KV cache sizes are computed using a toy configuration (2 layers, 2 heads, head dimension 32, FP16). A production model (e.g., Llama-2 7B: 32 layers, 32 heads, head dimension 128) would increase sizes by $\sim 256\times$, shifting absolute capacity dynamics but not relative policy comparisons.

Simulation. 6-hour continuous window; token counts clamped to 8,192 to preserve heavy-tailed distributions.

Capacity. 500MB total (50MB hot + 150MB warm + 300MB cold).

GPU Savings. Each hit is credited with the exact $O(N^2)$ recompute cost of the tokens *actually* cached (not the grown context at resume time), minus the modeled cost of restoring the entry from the tier where it was found (NVMe: 10ms + size/2GB/s; object storage: 100ms + size/500MB/s). Hits are therefore tier-discounted: a cold-tier hit is worth less than a hot-tier hit.

Statistics. Every (policy, workload) cell is run over 10 independent seeds. All policies replay the identical trace per seed, so policy comparisons are *paired*: we report per-seed deltas against LRU with 95% t-based confidence intervals across seeds.

9 Comparative Evaluation

We evaluated 6 policies across 3 workloads over 10 seeds. The casual workload achieved 100% hit rate at 500MB—the cache was never constrained—so Table 2 focuses on the two workloads where eviction pressure forces policy differentiation.

Table 2: Policy comparison (500MB, 6hr simulation, 10 seeds; mean with 95% t-CI across seeds; $\Delta\$$ is the *paired* per-day value delta vs. LRU on identical traces). Casual workload omitted: all policies reach 100% hit rate unconstrained.

Policy	Workload	Hit%	\$/Day	$\Delta\$$ vs LRU [CI]
LRU	Enterprise	79.2	2,723	—
Heuristic	Enterprise	55.3	2,467	−255 [−312, −199]
Logistic V1	Enterprise	67.9	2,492	−230 [−323, −138]
ValDensity V2	Enterprise	47.9	2,465	−257 [−348, −167]
ValDens+AC	Enterprise	47.9	2,465	−258 [−348, −167]
SpaceTime V3	Enterprise	51.2	2,559	−163 [−249, −77]
LRU	Power User	96.0	1,649	—
Heuristic	Power User	84.2	1,608	−41 [−60, −22]
Logistic V1	Power User	83.9	1,435	−214 [−276, −152]
ValDensity V2	Power User	71.9	1,447	−202 [−252, −152]
ValDens+AC	Power User	71.8	1,447	−202 [−252, −152]
SpaceTime V3	Power User	74.8	1,481	−168 [−215, −120]

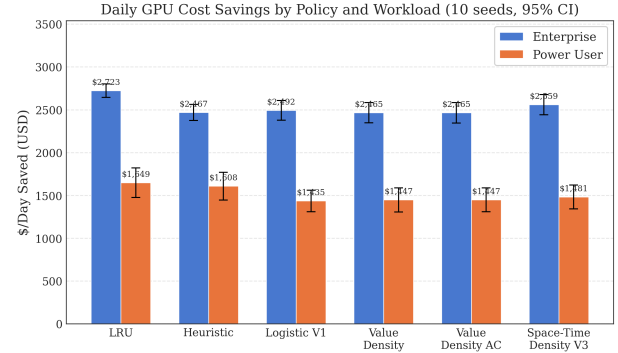


Figure 2: Daily GPU cost savings with 95% CIs. Note the compression of value gaps relative to hit-rate gaps on the power-user workload.

9.1 LRU Is a Stronger Baseline Than Assumed

Under honest evaluation—virtual-clock replay, cached-token crediting, tier-discounted hits, paired multi-seed statistics—**LRU wins both hit rate and dollar value on every constrained workload**, and every alternative’s paired value deficit is statistically significant. We emphasize this because an earlier version of our own pipeline, which replayed traces at wall-clock speed and credited hits with unclamped resume-time token counts, showed a learned policy *beating* LRU by \$68/day. That result was an artifact: collapsing simulated time zeroed the temporal features the model was trained on, and the accounting credited a different quantity than policies optimized. We report this reversal as a cautionary result for the learned-caching literature: **evaluation-harness bugs preferen-**

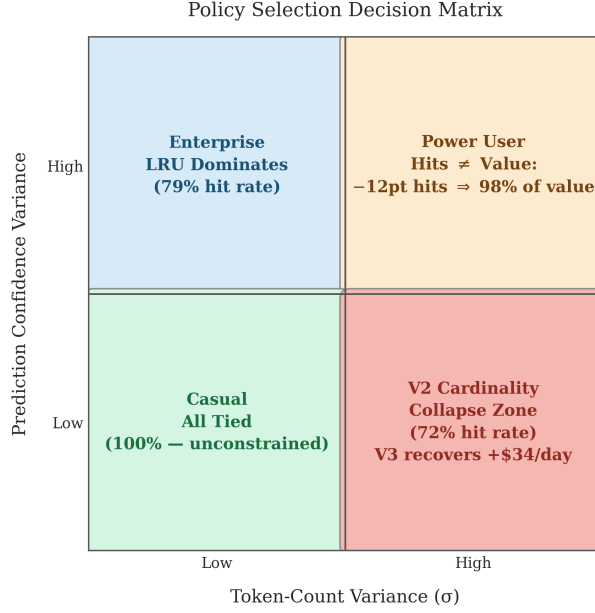


Figure 3: Policy selection decision matrix mapping failure modes by workload variance characteristics.

tially flatter complex policies, because simple recency policies are insensitive to broken time.

9.2 Why Static Knapsack Fails—and Lifetime Normalization Helps

V2 Value Density drops to 47.9% hit rate on enterprise and 71.9% on power-user, the worst of all policies. The Fractional Knapsack formulation is optimal for *static* packing, but a cache is *dynamic* under Little’s Law: by maximizing value density, V2 retains massive sessions whose sojourn time is unbounded, starving cache cardinality. Admission control (V2+AC) produces no improvement—under sustained load the cache operates at near-full utilization regardless of filtering.

The space-time correction (V3) directly tests the Little’s Law diagnosis: dividing the same value density by $\mathbb{E}[\Delta t]$ recovers **+\$94/day over V2 on enterprise** (paired 95% CI [+77, +112]) and **+\$34/day on power-user** [+20, +49], with hit rate improvements of ~ 3 points on both. The theory is validated directionally—lifetime normalization is the right correction—even though V3 still trails LRU. Closing that remaining gap is the open problem.

9.3 Hit Rate and Value Decouple

Hit rate remains the wrong lens even where LRU wins. On the power-user workload, the heuristic policy concedes **11.9 hit-rate points** to LRU (84.2% vs 96.0%) yet retains

Table 3: TinyLlama integration results (598-token prompt, CPU)

Metric	Trial 1	Trial 2	Average
Cold-start TTFT	11,804ms	11,449ms	11,627ms
Warm-start TTFT	239ms	243ms	241ms
TTFT speedup	49.4 $\times$	47.1 $\times$	48.3 $\times$
E2E speedup (30 tok.)	1.80 $\times$	1.69 $\times$	1.75 $\times$
Cache size (FP16)	13,156 KB	13,156 KB	13,156 KB
Save latency	55ms	57ms	56ms
Load latency	4ms	5ms	4.5ms
Semantic match	✓	✓	100%

97.5% of its dollar value (−\$41/day of \$1,649): the hits it gives up are disproportionately cheap-to-recompute sessions. Rankings also invert between metrics: Logistic V1 beats SpaceTime V3 by 9.1 hit-rate points on power-user, yet delivers **\$46/day less value** (paired CI [+25, +68] in V3’s favor). A researcher selecting policies by hit rate would choose backwards.

The correct optimization and evaluation target for KV-cache eviction is not hit rate—it is expected cache value.

10 Prototype Validation: TinyLlama

To validate with a real transformer, we integrated our `TieredCacheManager` with TinyLlama-1.1B-Chat (22 layers, 32 Q-heads, 4 KV-heads via GQA, head_dim=64) on CPU.

GQA Compatibility. Modern models use Grouped Query Attention where KV heads (4) differ from query heads (32). Our serialization must use `num_key_value_heads` for tensor dimensions—a critical detail for production deployment.

Warm-start TTFT of **241ms** versus cold-start 11,627ms represents a **48 $\times$ improvement**. End-to-end speedup is 1.75 $\times$ because decode-phase speed is independent of cache state. Greedy-decoded output was byte-identical, confirming FP16 quantization introduces no degradation.

Cost Model Calibration. Our simulation cost model (A100 GPU, \$2.50/hr) predicted 57,747ms for 598-token prefill versus actual 11,627ms on CPU—expected since model parameters must be calibrated per hardware. Relative policy comparisons remain valid.

Table 4: Break-even context length N^* (A100, $\eta=0.45$; NVMe: 10ms + 2GB/s; object storage: 100ms + 500MB/s). Persistence wins on latency beyond N^* .

Model	KV/token	N^* (NVMe)	N^* (S3)
TinyLlama-1.1B (GQA 4/32)	45 KB	1,561	25,901
Llama-2-7B (MHA 32/32)	512 KB	43,559	254,203
Llama-2-70B (GQA 8/64)	320 KB	12	289

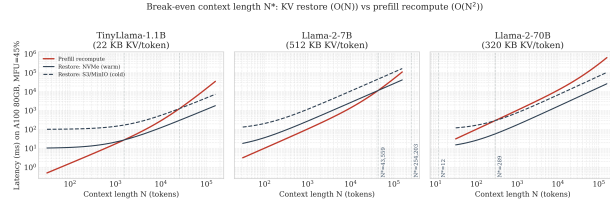


Figure 4: Prefill recompute ($O(N^2)$, red) vs. tier restore ($O(N)$, black) latency by context length. The crossover N^* varies by four orders of magnitude across model architectures.

11 When Does Persistence Beat Recompute? A Break-Even Analysis

The $48\times$ CPU speedup above should not be extrapolated to GPUs uncritically: on an A100, prefilling a 598-token prompt through a 1.1B model takes tens of milliseconds—comparable to the restore path itself. Persistence wins on raw latency only beyond a *break-even context length* N^* , which we can derive analytically.

Restoring a persisted cache moves $O(N)$ bytes:

$$t_{\text{restore}}(N) = \ell_{\text{tier}} + \frac{2 \cdot L \cdot H_{kv} \cdot d \cdot b \cdot N}{B_{\text{tier}}} \quad (3)$$

where ℓ is tier latency, B bandwidth, L layers, H_{kv} KV heads, d head dimension, b bytes per element. Recomputing the prefill costs $O(N)$ weight FLOPs plus $O(N^2)$ attention:

$$t_{\text{prefill}}(N) = \frac{2PN + 4LH_qdN^2}{F \cdot \eta} \quad (4)$$

with P parameters, H_q query heads (GQA shrinks the cache, not the score computation), F peak FLOP/s, and η the achievable utilization. Since restore is linear and prefill superlinear, persistence always wins beyond the crossover N^* .

Table 4 and Figure 4 expose the governing quantity: the ratio of KV bytes per token to prefill FLOPs per token. For MHA models like Llama-2-7B, the KV cache is so large relative to compute that NVMe restore only beats prefill past $\sim 43k$ tokens—persistence

is a *GPU-occupancy* optimization there, not a latency one. For large GQA models like Llama-2-70B, restore wins almost immediately ($N^*=12$ from NVMe): grouped attention shrinks bytes moved while parameters inflate FLOPs avoided. As the industry converges on GQA/MQA architectures, tiered KV persistence shifts from “sometimes useful” to near-universally latency-positive. (Analysis and figure reproducible via `benchmarks/breakeven_analysis.py`.)

12 Discussion & Future Work

The Metric Mismatch. Hit rate assumes uniform item value. In KV caching with quadratic recompute costs this is catastrophically wrong: hit-rate gaps of 12 points can correspond to value gaps of 2.5%, and policy rankings invert between the two metrics. Research and production systems must adopt value-weighted metrics (GPU-seconds saved per decision, discounted by restore cost).

Why does LRU win? Three compounding reasons. First, our workloads exhibit strong temporal locality—the majority of resumes occur within an hour of the previous access—which is precisely LRU’s inductive bias, obtained for free without a trained model. Second, value-aware policies pay a structural penalty under Little’s Law that lifetime normalization only partially removes. Third, LRU is *robust to evaluation bugs and distribution shift*: it consumes only access order, while learned policies consume nine features whose scales must match training exactly. We suspect published wins for learned eviction that do not control replay-time semantics deserve re-examination.

Decision Framework. (1) Uniform workloads ($\sigma=0.5$): LRU dominates—ML overhead is unjustified. (2) High-variance ($\sigma=1.2$): if the deployment metric is dollar value, simple recency heuristics are within noise of LRU and far cheaper than learned policies; evaluate by \$/Day, never hit rate. (3) Value-aware eviction must be lifetime-normalized (V3), not static (V2)—but neither yet beats LRU.

Limitations. (1) Synthetic workloads, though personally calibrated, may miss real user correlations; replaying production traces (e.g., Azure LLM inference traces) is the highest-priority extension. (2) CPU-only TinyLlama validation omits GPU-specific effects (CUDA fragmentation, PCIe overhead); the break-even model (§11) bounds but does not measure them. (3) Tier restore costs are modeled, not measured on production hardware. (4) The toy KV geometry preserves relative policy comparisons but not absolute capacity dynamics.

Beating LRU. V3’s remaining gap suggests two directions: hybrid policies that default to recency and deviate only for entries in the extreme value tail, where the

quadratic payoff dominates; and admission control at the *cold-to-warm promotion* boundary rather than at eviction time, where the Little’s Law penalty does not apply.

13 Conclusion

We presented a three-tier KV cache persistence system and a statistically rigorous evaluation of six eviction policies across three workloads and ten seeds, validated end-to-end with TinyLlama-1.1B. Our investigation yielded four insights: (1) under honest evaluation, LRU beats every learned and value-aware policy we tested on both metrics—and our own earlier harness bugs had shown the opposite, a caution for the learned-caching literature; (2) hit rate and dollar value decouple on heavy-tailed workloads, to the point of inverting policy rankings; (3) static value-density eviction collapses under Little’s Law, and lifetime normalization (space-time density, an LHD adaptation) significantly recovers value; (4) an analytic break-even model shows tiered persistence is latency-positive almost immediately for large GQA models. Prototype validation demonstrates $48\times$ CPU TTFT improvement with byte-identical restoration. Beating LRU on value remains open; we believe hybrid recency/value-tail policies are the most promising path.

References

- [1] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. LHD: Improving cache hit rate by maximizing hit density. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018.
- [2] László A. Bélády. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [3] Pei Cao and Sandy Irani. Cost-aware WWW proxy caching algorithms. In *Proceedings of the USENIX Symposium on Internet Technologies and Systems (USITS)*, 1997.
- [4] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (ATC)*, 2024.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [6] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [7] LMCache Team. LMCache: Accelerating LLM serving with reusable KV caches. GitHub Repository, 2024. URL <https://github.com/LMCache/LMCache>.
- [8] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*, 2025.
- [9] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Bélády for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [10] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.
- [11] Neal Young. On-line file caching. In *Proceedings of the 5th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 82–86, 1994.
- [12] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.